

fastEmbedR: native UMAP and openTSNE workflows

Moussa Kassim Martin Ocharo Dalia Ahmed Dupe Ojo Alessia Vignoli
 Leonardo Tenori Stefano Cacciatore

Contents

1	Overview	1
2	A first analysis	2
3	Choosing between UMAP and openTSNE	4
4	Graph clustering	4
5	PCA and reproducible openTSNE initialization	4
6	Reusing a nearest-neighbor graph	5
6.1	Independent UMAP initialization	6
7	UMAP graph modes	7
8	Float32 and memory-conscious analysis	8
9	Embedding new observations	8
10	Landmarking	10
11	Measuring embedding quality	12
12	CPU, Metal, and CUDA	13
13	A practical analysis checklist	14
14	References	14

1 Overview

`fastEmbedR` provides native implementations of two widely used nonlinear dimension-reduction workflows:

- UMAP from a sparse nearest-neighbor graph;
- openTSNE-style t-SNE with sparse attractive forces and an FFT-grid approximation for repulsion.

The same R interface selects native CPU, Apple Metal, or NVIDIA CUDA code. One-call functions include nearest-neighbor search, initialization, graph or affinity construction, and optimization. KNN-input functions let an existing neighbor graph be reused without repeating search. Explicit GPU requests either run on the requested GPU backend or fail with an informative error; they are never silently reported as GPU work after falling back to CPU.

The public workflow is intentionally small:

Goal	Function
Precompute native non-self neighbors	<code>precompute_knn()</code>
Run UMAP from a data matrix	<code>umap()</code>
Run openTSNE-style t-SNE from a data matrix	<code>opentsne()</code>
Reuse a KNN graph	<code>umap_knn()</code> or <code>opentsne_knn()</code>
Compute reusable PCA scores or t-SNE initialization	<code>pca()</code>
Accelerate a very large run with an explicit landmark approximation	<code>landmark_umap()</code> or <code>landmark_tsne()</code>
Place new observations in a fixed t-SNE reference	<code>transform_tsne()</code>
Quantify embedding quality	<code>evaluate_embedding()</code>
Build a compact clustering graph	<code>knn_graph()</code>
Find Louvain, Leiden, or Walktrap communities	<code>graph_cluster()</code>

2 A first analysis

We use the four numeric Iris measurements for a small, reproducible example. The matrix is scaled once outside the embedding calls, so `standardize = FALSE` prevents repeated preprocessing.

```
library(fastEmbedR)

x <- scale(as.matrix(iris[, 1:4]))
labels <- iris$Species
dim(x)
#> [1] 150  4
```

UMAP uses a neighborhood size, whereas t-SNE uses perplexity. For this small dataset we use 15 neighbors and perplexity 10. Missing openTSNE optimization settings are chosen by the package's native auto-configuration policy. Its default initialization is a small-scale randomized-PCA layout.

```
fit_umap <- umap(
  x,
  n_neighbors = 15,
  graph_mode = "fuzzy",
  standardize = FALSE,
  n_threads = 2,
  seed = 4
)

fit_tsne <- opentsne(
  x,
  perplexity = 10,
  standardize = FALSE,
  n_threads = 2,
  seed = 4
)

c(
  UMAP_seconds = fit_umap$metrics$elapsed,
  openTSNE_seconds = fit_tsne$metrics$elapsed
)
#>      UMAP_seconds openTSNE_seconds
#>      0.102      0.420
```

Both functions return a `fastEmbedR_embedding` object. The full coordinates are in `$layout`; `$metrics`,

\$timings, and \$parameters preserve enough context to audit a run.

```
old_par <- par(mfrow = c(1, 2), mar = c(1, 1, 2.5, 1), bg = "white")
plot_embedding(fit_tsne$layout, labels, "openTSNE")
plot_embedding(fit_umap$layout, labels, "UMAP (fuzzy graph)")
```

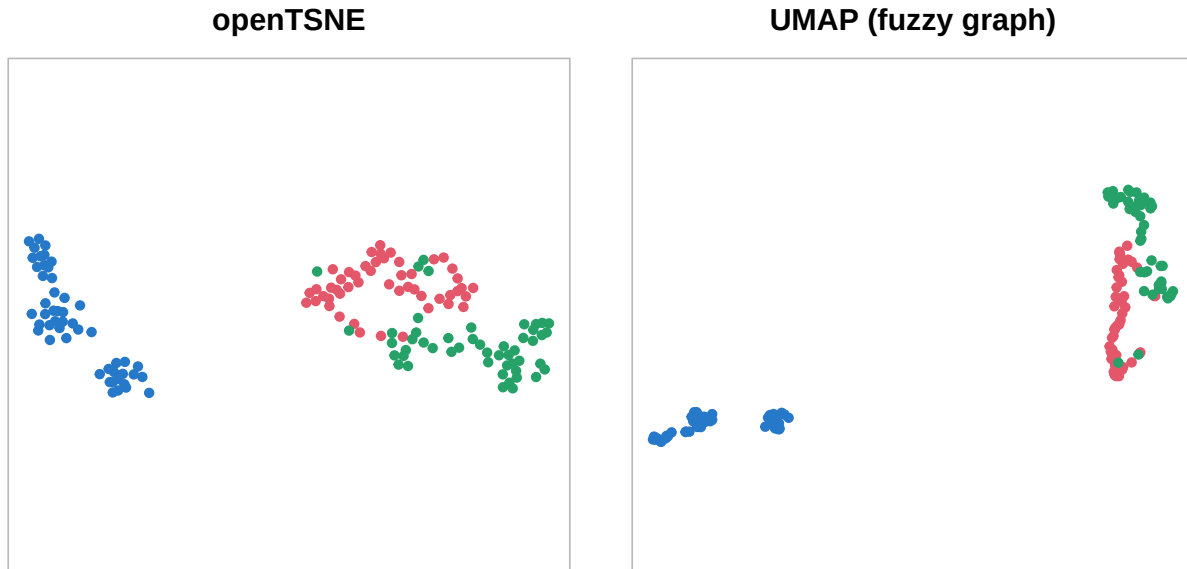


Figure 1: Iris embeddings produced by the one-call CPU workflows.

```
par(old_par)
```

The backend recorded in the result is the backend that actually ran:

```
fit_umap$parameters[c("backend", "nn_backend", "n_threads", "graph_mode")]
#> $backend
#> [1] "cpu"
#>
#> $nn_backend
#> [1] "cpu"
#>
#> $n_threads
#> [1] 2
#>
#> $graph_mode
#> [1] "fuzzy"
fit_tsne$parameters[c("backend", "nn_backend", "n_threads", "perplexity")]
#> $backend
#> [1] "cpu"
#>
#> $nn_backend
#> [1] "cpu"
#>
#> $n_threads
#> [1] 2
#>
```

```
#> $perplexity
#> [1] 10
```

3 Choosing between UMAP and openTSNE

The two methods answer related but different questions.

Property	UMAP	openTSNE-style t-SNE
High-dimensional model	Weighted or binary KNN graph	Sparse probability affinities
Main scale parameter	n_neighbors	perplexity
Initialization	Graph-native initialization	Randomized PCA by default
Repulsion	Sampled negative edges	Exact or FFT-grid negative gradient
New observations	Landmark interpolation/refinement	Fixed-reference t-SNE transform
Typical interpretation	Local neighborhoods with some graph-level organization	Local probability neighborhoods and cluster organization

Distances and empty space in either two-dimensional plot should not be treated as calibrated biological effect sizes. Compare methods with neighborhood and label-aware metrics rather than visual appearance alone.

4 Graph clustering

`knn_graph()` builds a compact undirected graph from raw data, an embedding, or precomputed neighbors. It uses the existing KNN route selected by **backend**; there is no second nearest-neighbor method menu inside the graph API.

```
graph <- knn_graph(fit_tsne, k = 20, weight = "snn", n_threads = 2)
communities <- graph_cluster(graph, method = "leiden", seed = 4)
table(communities$membership)
#>
#> 1 2 3 4 5 6
#> 33 17 33 21 17 29
```

Community detection is native C++ CPU code and is reported as such. Louvain and Leiden use sparse graph storage and scale to large KNN graphs. **walktrap** is the true Pons-Latapy random-walk agglomeration method; its exact transition matrix is intentionally limited to moderate graphs. Neither **igraph**, **cuGraph**, nor an external clustering routine is called at run time.

5 PCA and reproducible openTSNE initialization

`pca()` exposes the backend-native truncated PCA used for initialization. Its core argument order follows `fastPLS::pca()` (**x**, **ncomp**, **xtest**, **center**, **scale**, and **backend**) but deliberately omits an SVD-method menu. Randomized SVD is the supported decomposition family. `opentsne_init = FALSE` is the default; request it when the t-SNE-ready layout is needed.

```
pca_fit <- pca(
  x,
  ncomp = 2,
  center = TRUE,
  scale = FALSE,
```

```

backend = "cpu",
seed = 4,
opentsne_init = TRUE
)

c(
  score_rows = nrow(pca_fit$scores),
  score_columns = ncol(pca_fit$scores),
  init_max_sd = max(apply(as.matrix(pca_fit$opentsne_init), 2, stats::sd))
)
#>      score_rows score_columns  init_max_sd
#>      1.5e+02      2.0e+00      1.0e-04

```

The ordinary scores remain in `pca_fit$scores`. The added `pca_fit$opentsne_init` is centered and rescaled so the largest component standard deviation is $1e-4$, the small initialization scale expected by t-SNE optimizers. It can be reused across methods, seeds, or backends:

```

layout_tsne_fixed_init <- opentsne(
  x,
  perplexity = 10,
  Y_init = pca_fit$opentsne_init,
  standardize = FALSE,
  n_threads = 2,
  seed = 4
)

```

CPU PCA uses the package’s fastPLS-style RSVD path. Metal uses a native float32 block-subspace TSVD with Metal Performance Shaders matrix products. CUDA uses native RAPIDS RAFT TSVD when that support is compiled. No backend calls Python or `reticulate`.

6 Reusing a nearest-neighbor graph

The KNN-input API is useful when comparing embeddings, graph modes, or seeds. `precompute_knn()` exposes the same backend-specific native search selected by the one-call embedding functions, without exposing a second algorithm menu.

```

knn <- precompute_knn(
  x,
  k = 15,
  metric = "euclidean",
  backend = "cpu",
  n_threads = 2
)
knn
#> <fastEmbedR_knn>
#>  observations: 150
#>  neighbors:    15 (non-self)
#>  metric:       euclidean
#>  backend:      cpu
#>  engine:       native_cpu_hnsw
#>  residency:    host
#>  elapsed:      0.004 s

```

`prepare_umap_knn()` builds graph state and `epochs_per_sample` once. `prepare_opentsne_knn()` validates, strips self-neighbors when needed, and stores a compact KNN width. Prepared objects are useful for repeated

runs.

```
prepared_umap <- prepare_umap_knn(
  knn,
  graph_mode = "fuzzy",
  n_threads = 2
)
prepared_tsne <- prepare_opentsne_knn(knn, perplexity = 10)

layout_umap_reused <- umap_knn(
  prepared_umap,
  n_threads = 2,
  seed = 4
)
layout_tsne_reused <- opentsne_knn(
  prepared_tsne,
  Y_init = pca_fit$opentsne_init,
  n_threads = 2,
  seed = 4
)

cbind(UMAP = dim(layout_umap_reused), openTSNE = dim(layout_tsne_reused))
#>      UMAP openTSNE
#> [1,]  150      150
#> [2,]   2         2
```

6.1 Independent UMAP initialization

`umap_init()` separates graph construction and sparse initialization from layout optimization. The returned object retains the exact prepared graph and initial coordinates, so `umap_knn()` can reuse both without rebuilding either stage. This is useful for repeated seeds, backend agreement tests, and optimizer-only timing.

```
umap_initialization <- umap_init(
  prepared_umap,
  backend = "cpu",
  graph_mode = "fuzzy",
  n_threads = 2,
  seed = 4
)

layout_from_initialization <- umap_knn(
  umap_initialization,
  backend = "cpu",
  n_threads = 2,
  seed = 4
)

umap_initialization$timings
#>      stage elapsed_sec
#> 1      knn      0.000
#> 2     graph      0.000
#> 3 initialization    0.002
#> 4      total      0.002
```

The initialization object can also be passed to a Metal or CUDA optimizer. That controlled comparison holds

the graph and starting coordinates fixed; ordinary one-call GPU embeddings continue to use their resident backend path.

7 UMAP graph modes

The graph mode is an explicit scientific choice, not a hidden speed setting.

- `graph_mode = "fuzzy"` uses locally scaled fuzzy membership strengths from the standard UMAP graph construction.
- `graph_mode = "binary"` symmetrizes unit-weight neighbor edges and then uses the same UMAP optimizer. It is the package default and can produce stronger separation on some datasets.

```
layout_binary <- umap_knn(  
  knn,  
  graph_mode = "binary",  
  n_threads = 2,  
  seed = 4  
)  
layout_fuzzy <- umap_knn(  
  knn,  
  graph_mode = "fuzzy",  
  n_threads = 2,  
  seed = 4  
)  
  
old_par <- par(mfrow = c(1, 2), mar = c(1, 1, 2.5, 1), bg = "white")  
plot_embedding(layout_binary, labels, "Binary graph")  
plot_embedding(layout_fuzzy, labels, "Fuzzy graph")
```

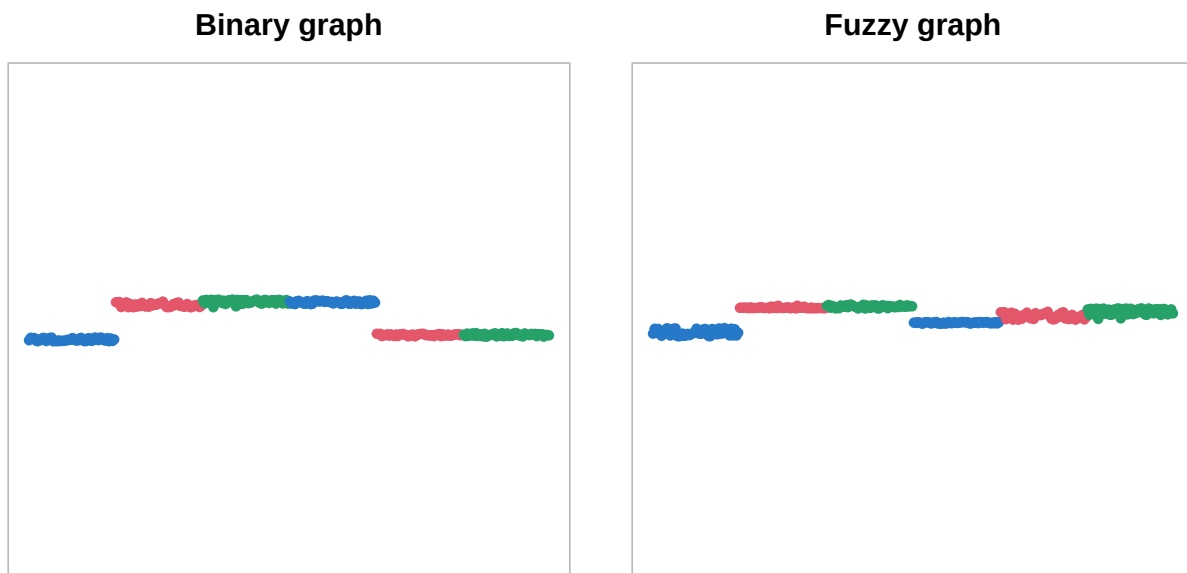


Figure 2: The UMAP graph mode changes edge weights while retaining the same optimizer.

```
par(old_par)
```

Record the chosen mode whenever embeddings are reported or compared.

8 Float32 and memory-conscious analysis

All native embedding buffers use float32 and integer indices. A standard R numeric matrix is converted at the native boundary. Supplying a `float::float32` matrix avoids retaining an unnecessary double-precision copy in R, and the returned layout remains float32.

```
if (requireNamespace("float", quietly = TRUE)) {
  x_float <- float::f1(x)
  fit_float <- umap(
    x_float,
    n_neighbors = 15,
    graph_mode = "fuzzy",
    standardize = FALSE,
    n_threads = 2,
    seed = 4
  )
  c(
    input_class = class(x_float)[1],
    layout_class = class(fit_float$layout)[1]
  )
}
#> input_class layout_class
#> "float32" "float32"
```

For large datasets:

1. perform scaling or feature extraction once and reuse the result;
2. use float32 input when practical;
3. leave `keep_knn = FALSE` unless neighbor matrices are needed later;
4. use prepared KNN/graph objects for repeated seeds;
5. consider landmarking only when its approximation is scientifically acceptable.

9 Embedding new observations

`transform_tsne()` places query observations into an existing reference embedding without moving the reference points. It builds query-to-reference affinities, initializes each query near its reference neighbors, and optimizes only the query coordinates. Reference and query data must have undergone the same preprocessing.

For this executable example, every fifth Iris observation is treated as new. We compute its exact query-to-reference KNN directly. The same API accepts any plain host list with query-to-reference `indices` and `distances`.

```
query_rows <- seq(5L, nrow(x), by = 5L)
reference_rows <- setdiff(seq_len(nrow(x)), query_rows)
reference_data <- x[reference_rows, , drop = FALSE]
query_data <- x[query_rows, , drop = FALSE]

reference_fit <- opentsne(
  reference_data,
  perplexity = 10,
  standardize = FALSE,
  n_threads = 2,
  seed = 4
)

exact_query_knn <- function(reference, query, k) {
```

```

squared_distances <- outer(
  rowSums(query^2), rowSums(reference^2), "+"
) - 2 * tcrossprod(query, reference)
squared_distances[squared_distances < 0] <- 0
indices <- t(apply(squared_distances, 1L, order))[
  , seq_len(k), drop = FALSE
]
distances <- matrix(
  sqrt(squared_distances[
    cbind(rep(seq_len(nrow(query)), each = k), as.vector(t(indices)))
  ]),
  nrow = nrow(query),
  byrow = TRUE
)
list(indices = indices, distances = distances)
}

query_knn <- exact_query_knn(reference_data, query_data, k = 15)
query_layout <- transform_tsne(
  reference_fit,
  knn = query_knn,
  perplexity = 5,
  n_iter = 100,
  n_threads = 2,
  backend = "cpu",
  seed = 4
)

reference_labels <- factor(labels[reference_rows], levels = levels(labels))
query_labels <- factor(labels[query_rows], levels = levels(labels))
par(bg = "white")
plot(
  reference_fit$layout,
  col = embedding_colors[as.integer(reference_labels)],
  pch = 19,
  cex = 0.65,
  asp = 1,
  axes = FALSE,
  xlab = "",
  ylab = "",
  main = "Fixed-reference transform"
)
points(
  query_layout,
  col = embedding_colors[as.integer(query_labels)],
  pch = 17,
  cex = 1.05
)
legend(
  "bottomright",
  legend = c("Reference", "New"),
  pch = c(19, 17),
  bty = "n",
  cex = 0.8
)

```

```
)
box(col = "#B8B8B8")
```

Fixed-reference transform

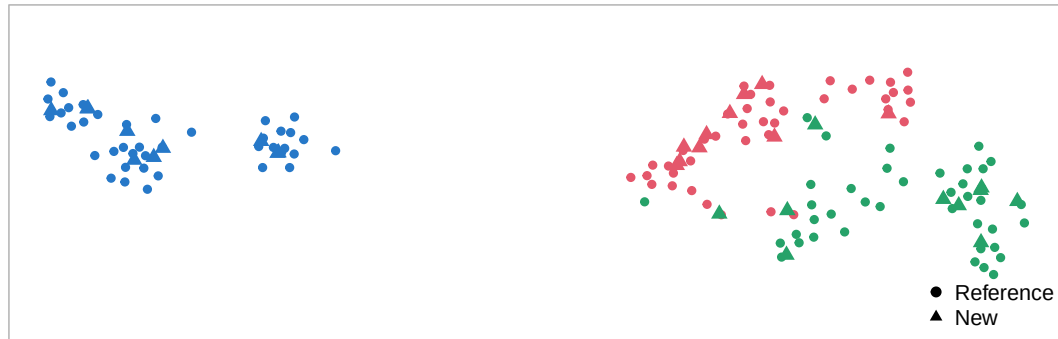


Figure 3: New observations (triangles) transformed into a fixed openTSNE reference (circles).

For Metal or CUDA, request the same backend explicitly. If the native fixed-reference path is unavailable, the call fails rather than relabeling a CPU transform as GPU work.

10 Landmarking

Landmarking embeds a representative subset and places the remaining rows in that reference. It is never activated silently by `umap()` or `opentsne()`. The final layout still has one row per input observation. Reference and projection searches use fastEmbedR's native KNN routes.

The staged API keeps selection, reference fitting, and projection separate. This is useful when the same landmark reference will be reused:

```
landmark_selection <- select_landmarks(
  x,
  landmarks = 0.5,
  seed = 4,
  n_threads = 2
)

umap_reference <- fit_landmark_model(
  x,
  landmark_selection,
  method = "umap",
  n_neighbors = 15,
  graph_mode = "fuzzy",
  backend = "cpu",
  n_threads = 2,
  seed = 4
```

```
)

umap_projected <- project_landmark_model(
  umap_reference,
  x,
  refinement_epochs = 50,
  n_threads = 2
)
```

`fit_landmark_model()` invokes the ordinary production `umap()` or `opentsne()` implementation on the reference rows. Consequently, UMAP `graph_mode`, optimizer settings, seed, and backend are explicit and consistent with a full-data fit. `project_landmark_model()` computes only the query-to-reference neighbors. It does not recompute neighbors among landmarks or among query observations. Passing a matrix of new observations instead of `x` projects those observations into the same fixed reference embedding.

The one-call functions below perform the same workflow when the intermediate selection and model do not need to be retained.

```
fit_landmark_umap <- landmark_umap(
  x,
  landmarks = 0.5,
  n_neighbors = 15,
  graph_mode = "fuzzy",
  standardize = FALSE,
  n_threads = 2,
  seed = 4
)

fit_landmark_tsne <- landmark_tsne(
  x,
  landmarks = 0.5,
  perplexity = 10,
  standardize = FALSE,
  n_threads = 2,
  seed = 4
)

rbind(
  UMAP = fit_landmark_umap$metrics[
    1, c("elapsed", "reference_embedding_elapsed",
        "landmark_projection_knn_elapsed", "landmark_refinement_elapsed")
  ],
  openTSNE = setNames(
    fit_landmark_tsne$metrics[
      1, c("elapsed", "reference_embedding_elapsed",
          "landmark_projection_knn_elapsed", "transform_elapsed")
    ],
    c("elapsed", "reference_embedding_elapsed",
      "landmark_projection_knn_elapsed", "landmark_refinement_elapsed")
  )
)

#>           elapsed reference_embedding_elapsed landmark_projection_knn_elapsed
#> UMAP           0.175                      0.171                      0.002
#> openTSNE       0.095                      0.089                      0.001
#>           landmark_refinement_elapsed
```

```
#> UMAP                                0.002
#> openTSNE                             0.005

old_par <- par(mfrow = c(1, 2), mar = c(1, 1, 2.5, 1), bg = "white")
plot_embedding(fit_landmark_tsne$layout, labels, "Landmark openTSNE")
plot_embedding(fit_landmark_umap$layout, labels, "Landmark UMAP")
```

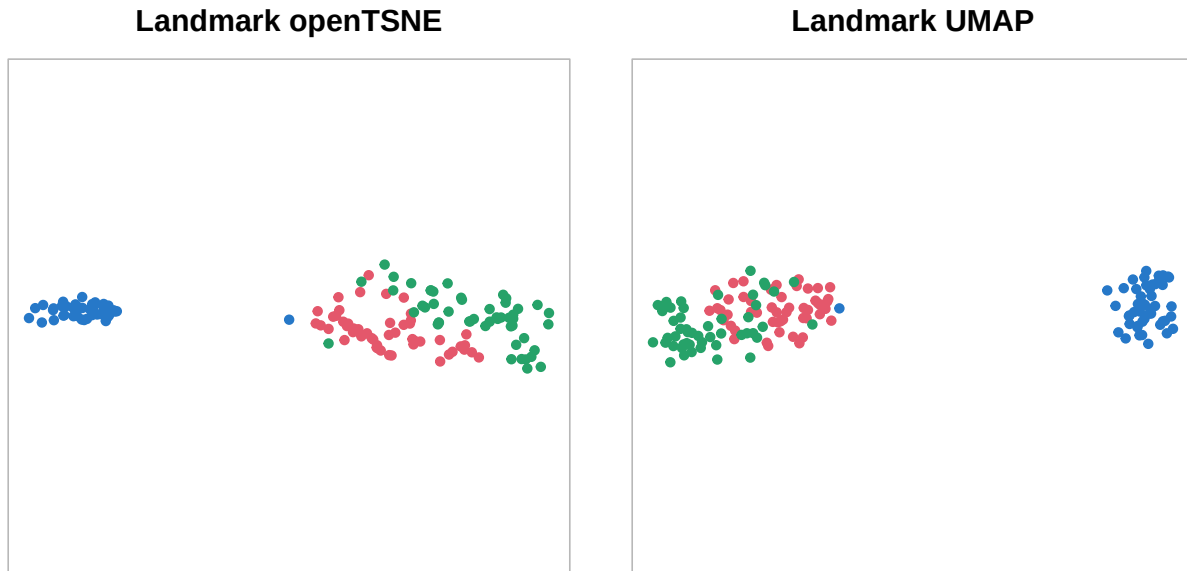


Figure 4: Iris embeddings using 50% of observations as landmarks.

```
par(old_par)
```

CPU uses native HNSW for query-to-reference search. Metal uses native exact search for small references and recall-tuned IVF-Flat for larger references, followed by fixed-reference transforms. CUDA uses native exact or IVF-Flat query search and keeps KNN, layouts, and refinement buffers resident on the GPU. The result records landmark count, fraction, projection backend, and separate reference, query-KNN, and transform timings so a landmark approximation cannot be mistaken for a full embedding.

11 Measuring embedding quality

`evaluate_embedding()` reports local, global, and label-aware metrics. For large datasets it uses deterministic subsamples rather than constructing a complete all-pairs distance matrix. Here all Iris observations are small enough to evaluate directly. Supplying `reference_nn` also avoids repeating the high-dimensional neighbor search.

```
quality_umap <- evaluate_embedding(
  x,
  fit_umap$layout,
  labels = labels,
  k = 10,
  primary_k = 10,
  reference_nn = knn,
  sample_size_for_local_metrics = nrow(x),
  sample_size_for_global_metrics = nrow(x),
```

```

seed = 4,
method = "UMAP",
backend = "cpu",
dataset = "iris",
n_threads = 2
)

quality_tsne <- evaluate_embedding(
  x,
  fit_tsne$layout,
  labels = labels,
  k = 10,
  primary_k = 10,
  reference_nn = knn,
  sample_size_for_local_metrics = nrow(x),
  sample_size_for_global_metrics = nrow(x),
  seed = 4,
  method = "openTSNE",
  backend = "cpu",
  dataset = "iris",
  n_threads = 2
)

quality <- rbind(quality_umap, quality_tsne)
quality[, c(
  "method", "trustworthiness", "knn_preservation_10",
  "distance_spearman", "silhouette", "label_knn_accuracy"
)]
#>      method trustworthiness knn_preservation_10 distance_spearman silhouette
#> 1      UMAP      0.9745576      0.7226667      0.8969430 0.5223689
#> 2 openTSNE      0.9887584      0.8060000      0.9168283 0.4897191
#>      label_knn_accuracy
#> 1      0.9533333
#> 2      0.9466667

```

No single score is sufficient. Trustworthiness and neighbor preservation measure local structure; distance correlation is a coarse global diagnostic; silhouette and KNN label accuracy are meaningful only when labels are valid external annotations rather than tuning targets.

12 CPU, Metal, and CUDA

The backend argument has exactly three choices.

Backend	One-call KNN	Embedding implementation
"cpu"	Native float32 HNSW	Native multithreaded C++ UMAP or FFT-grid openTSNE
"metal"	Native exact or recall-tuned IVF-Flat	Objective-C++/Metal kernels; no Python bridge
"cuda"	Native FAISS GPU exact search or RAPIDS cuVS IVF-Flat	CUDA UMAP or CUDA/cuFFT openTSNE with resident device buffers

GPU use is explicit:

```
fit_metal <- opentsne(x, perplexity = 10, backend = "metal", seed = 4)
fit_cuda <- umap(x, n_neighbors = 15, backend = "cuda", seed = 4)
```

If the requested backend was not compiled or is unavailable at runtime, these calls stop. CPU fallback is not relabeled as Metal or CUDA. Because parallel reductions and atomic updates can be ordered differently, fixed seeds provide controlled stochastic initialization and sampling but do not promise bitwise identity between devices or thread counts.

13 A practical analysis checklist

Before publishing an embedding analysis:

1. state whether input features were centered, scaled, or reduced by PCA;
2. report method, backend, seed, `n_neighbors` or perplexity, and UMAP graph mode;
3. distinguish full runs from landmark approximations;
4. compare total runtime when competing packages expose different internal boundaries;
5. report trustworthiness and neighbor preservation alongside any label-aware score;
6. inspect the plot for outliers, splitting, crowding, and seed sensitivity;
7. save `sessionInfo()` and the result's `$parameters` and `$timings`.

14 References

1. van der Maaten L, Hinton G. Visualizing data using t-SNE. *J Mach Learn Res* 2008;9:2579-2605.
2. Policar PG, Strazar M, Zupan B. openTSNE: a modular Python library for t-SNE dimensionality reduction and embedding. *J Open Source Softw* 2019;4:1576.
3. Linderman GC, Rachh M, Hoskins JG, Steinerberger S, Kluger Y. Fast interpolation-based t-SNE for improved visualization of single-cell RNA-seq data. *Nat Methods* 2019;16:243-245.
4. McInnes L, Healy J, Melville J. UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv* 2018;1802.03426.
5. Belkina AC, Ciccolella CO, Anno R, Halpert R, Spidlen J, Snyder-Cappione JE. Automated optimized parameters for T-distributed stochastic neighbor embedding improve visualization and analysis of large datasets. *Nat Commun* 2019;10:5415.
6. Policar PG, Stražar M, Zupan B. Embedding to reference t-SNE space addresses batch effects in single-cell classification. *Mach Learn* 2021;110:3395-3417.
7. Johnson J, Douze M, Jegou H. Billion-scale similarity search with GPUs. *IEEE Trans Big Data* 2021;7:535-547.

See the companion *Algorithmic design and acknowledgements* vignette for the native implementation, performance engineering, software provenance, and license boundaries.

```
sessionInfo()
#> R version 4.6.0 (2026-04-24)
#> Platform: aarch64-apple-darwin23
#> Running under: macOS Sonoma 14.5
#>
#> Matrix products: default
#> BLAS: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK version
#>
#> locale:
#> [1] C
#>
#> time zone: Africa/Johannesburg
```

```
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods   base
#>
#> other attached packages:
#> [1] fastEmbedR_0.99.0 knitr_1.51      rmarkdown_2.31
#>
#> loaded via a namespace (and not attached):
#> [1] digest_0.6.39 fastmap_1.2.0 Matrix_1.7-5  xfun_0.60      lattice_0.22-9
#> [6] float_0.3-3 parallel_4.6.0 htmltools_0.5.9 cli_3.6.6      fastPLS_0.99.6
#> [11] grid_4.6.0 withr_3.0.3 compiler_4.6.0 tools_4.6.0    evaluate_1.0.5
#> [16] Rcpp_1.1.2 yaml_2.3.12 otl_0.2.0     jsonlite_2.0.0 rlang_1.3.0
```